

QUESTIONS 10 / 10 completed

Trie Data Structure ▾

TRIE DATA STRUCTURE

"Implement a Trie Data Structure for 15 words. Perform 5 exact-word searches and 3 prefix searches.

Display:
Search path traversed.
Nodes visited during search.
Whether the word is found or not.

PROGRAM EDITOR

C ▾

Run

Save

```
1 #include <stdio.h>
2 #include <stdlib.h>
3 #include <string.h>
4
5 #define ALPHABET_SIZE 26
6
7 typedef struct TrieNode {
8     struct TrieNode *children[ALPHABET_SIZE];
9     int isEnd;
10 } TrieNode;
11
12 TrieNode* createNode() {
13     TrieNode *node = (TrieNode*)malloc(sizeof(TrieNode));
14     node->isEnd = 0;
15     for (int i = 0; i < ALPHABET_SIZE; i++)
16         node->children[i] = NULL;
17     return node;
18 }
19
20 void insert(TrieNode *root, char *word) {
21     TrieNode *curr = root;
```

INPUT

15
apple
app
apply
apt
bat
ball
banana
band
bank
cat
call
cap
dog
door
doll

QUESTIONS 10 / 10 completed

Trie Data Structure- ▾

TRIE DATA STRUCTURE-2

Write a C program to implement a Trie Data Structure that stores a set of words. The program should allow the user to search for a word and also check whether a given prefix exists in the Trie.

PROGRAM EDITOR

C ▾

Run

Save

```
1 #include <stdio.h>
2 #include <stdlib.h>
3 #include <string.h>
4
5 #define SIZE 26
6
7 typedef struct TrieNode {
8     struct TrieNode *children[SIZE];
9     int isEndOfWord;
10 } TrieNode;
11
12 // Create a new Trie node
13 TrieNode* createNode() {
14     TrieNode *node = (TrieNode*)malloc(sizeof(TrieNode));
15     node->isEndOfWord = 0;
16
17     for (int i = 0; i < SIZE; i++)
18         node->children[i] = NULL;
19
20     return node;
21 }
```

INPUT

5
apple
app
apply
bat
ball
apple
ap

QUESTIONS 10 / 10 completed

Search Suggestion ▾

SEARCH SUGGESTION SYSTEM

Write a C program to implement a search suggestion system using a Trie. For each character entered by the user, display all possible word suggestions from the Trie.

PROGRAM EDITOR

C ▾

Run

Save

```
1 #include <stdio.h>
2 #include <stdlib.h>
3 #include <string.h>
4
5 #define ALPHABET_SIZE 26
6
7 typedef struct TrieNode {
8     struct TrieNode *children[ALPHABET_SIZE];
9     int isEndOfWord;
10 } TrieNode;
11
12 TrieNode* createNode() {
13     TrieNode *node = (TrieNode*)malloc(sizeof(TrieNode));
14
15     for (int i = 0; i < ALPHABET_SIZE; i++)
16         node->children[i] = NULL;
17
18     node->isEndOfWord = 0;
19     return node;
20 }
21
```

INPUT

8
apple
app
apply
apt
bat
ball
banana
band
app

QUESTIONS 10 / 10 completed

Trie-Based Contact ▾

TRIE-BASED CONTACT DIRECTORY

"Trie-Based Contact Directory

Implement a contact management system using Trie.

Store 15 contact names and perform searches.

Display:

Contact insertion path.

Matching contacts for ▾

PROGRAM EDITOR

C ▾

Run

Save

```
1 #include <stdio.h>
2 #include <stdlib.h>
3 #include <string.h>
4
5 #define SIZE 26
6
7 typedef struct TrieNode {
8     struct TrieNode *children[SIZE];
9     int isEnd;
10 } TrieNode;
11
12 int totalFound = 0;
13
14 TrieNode* createNode() {
15     TrieNode *node = (TrieNode*)malloc(sizeof(TrieNode));
16     node->isEnd = 0;
17
18     for (int i = 0; i < SIZE; i++)
19         node->children[i] = NULL;
20
21     return node;
22 }
```

INPUT

al

QUESTIONS 10 / 10 completed

Trie Search & Prefix

TRIE SEARCH & PREFIX MATCHING

"Trie Search and Prefix Matching

Implement a Trie Data Structure for 15 words. Perform 5 exact-word searches and 3 prefix searches.

Display:
Search path traversed.
Nodes visited during search.

PROGRAM EDITOR

C

Run

Save

```
1 #include <stdio.h>
2 #include <stdlib.h>
3 #include <string.h>
4
5 #define ALPHABET_SIZE 26
6
7 typedef struct TrieNode {
8     struct TrieNode *children[ALPHABET_SIZE];
9     int isEndOfWord;
10 } TrieNode;
11
12 TrieNode* createNode() {
13     TrieNode *node = (TrieNode*)malloc(sizeof(TrieNode));
14
15     for (int i = 0; i < ALPHABET_SIZE; i++)
16         node->children[i] = NULL;
17
18     node->isEndOfWord = 0;
19     return node;
20 }
21
```

INPUT

apple app apply apt
bat
ball banana band
bank cat
call cap dog door doll

QUESTIONS 10 / 10 completed

Longest Common I ▾

LONGEST COMMON PREFIX

Longest Common Prefix Using Trie

Implement a Trie to find the longest common prefix among 10 strings.

Display:
Trie construction process.
Common path

PROGRAM EDITOR

C ▾

Run

Save

```
1 #include <stdio.h>
2 #include <stdlib.h>
3 #include <string.h>
4
5 #define ALPHABET_SIZE 26
6
7 typedef struct TrieNode {
8     struct TrieNode *children[ALPHABET_SIZE];
9     int childCount;
10    int isEndOfWord;
11 } TrieNode;
12
13 TrieNode* createNode() {
14     TrieNode *node = (TrieNode*)malloc(sizeof(TrieNode));
15
16     for (int i = 0; i < ALPHABET_SIZE; i++)
17         node->children[i] = NULL;
18
19     node->childCount = 0;
20     node->isEndOfWord = 0;
21 }
```

INPUT

flower
flow
flight
flock
flour
floor
flame
flash
flat
flag

QUESTIONS 10 / 10 completed

Trie-Spell Checker ▾

TRIE-SPELL CHECKER

Write a C program to implement a spell checker using a Trie. The program should verify whether a given word exists in the dictionary and suggest possible corrections based on matching prefixes.

PROGRAM EDITOR

C ▾

Run

Save

```
1 #include <stdio.h>
2 #include <stdlib.h>
3 #include <string.h>
4
5 #define SIZE 26
6
7 typedef struct TrieNode {
8     struct TrieNode *children[SIZE];
9     int isEndOfWord;
10 } TrieNode;
11
12 TrieNode* createNode() {
13     TrieNode *node = (TrieNode*)malloc(sizeof(TrieNode));
14
15     for (int i = 0; i < SIZE; i++)
16         node->children[i] = NULL;
17
18     node->isEndOfWord = 0;
19     return node;
20 }
21
```

INPUT

aple

QUESTIONS 10 / 10 completed

Trie-Menu-Driven

TRIE-MENU-DRIVEN

"Write a C program to implement a menu-driven Trie Data Structure supporting the following operations:
Insert a word
Search for a word
Delete a word
Display all stored words
Exit

PROGRAM EDITOR

C

Run

Save

```
1 #include <stdio.h>
2 #include <stdlib.h>
3 #include <string.h>
4
5 #define SIZE 26
6
7 typedef struct TrieNode {
8     struct TrieNode *children[SIZE];
9     int isEndOfWord;
10 } TrieNode;
11
12 TrieNode* createNode() {
13     TrieNode *node = (TrieNode*)malloc(sizeof(TrieNode));
14
15     for (int i = 0; i < SIZE; i++)
16         node->children[i] = NULL;
17
18     node->isEndOfWord = 0;
19     return node;
20 }
21
```

INPUT

1
apple
1
app
1
bat
4
2
apple
3
app
4
5

QUESTIONS 10 / 10 completed

Traversal and Lexicographical

TRAVERSAL AND LEXICOGRAPHICAL

"Trie Traversal and Lexicographical Ordering

Implement a Trie and store 20 words.

Display:

Trie structure.

Words in lexicographical order.

Total words stored.

Traversal path.

PROGRAM EDITOR

C



Run

Save

```
1 #include <stdio.h>
2 #include <stdlib.h>
3 #include <string.h>
4
5 #define SIZE 26
6
7 typedef struct TrieNode {
8     struct TrieNode *children[SIZE];
9     int isEndOfWord;
10 } TrieNode;
11
12 int totalWords = 0;
13 int nodesVisited = 0;
14
15 TrieNode* createNode() {
16     TrieNode *node = (TrieNode*)malloc(sizeof(TrieNode));
17
18     for (int i = 0; i < SIZE; i++)
19         node->children[i] = NULL;
20
21     node->isEndOfWord = 0;
```

INPUT

No input required